UNIQ ID: E0124092

Solving the Taxi Problem using Q Learning

Goal:

Say our agent is the driving the taxi. There are totally four locations and the agent has to pick up a passenger at one location and drop at the another. The agent will receive +20 points as a reward for successful drop off and -1 point for every time step it takes. The agent will also lose -10 points for illegal pickups and drops. So the goal of our agent is to learn to pick up and drop passengers at the correct location in a short time without boarding any illegal passengers.

First, we import all necessary libraries and simulate the environment

```
import random
import gym
env = gym.make('Taxi-v3')
```

```
Gym has been unmaintained since 2022 and does not support NumPy 2.0 amongst other critical functionality.
Please upgrade to Gymnasium, the maintained drop-in replacement of Gym, or contact the authors of your software and request that they upgrade.
See the migration guide at https://gymnasium.farama.org/introduction/migration_guide/ for additional information.
/usr/local/lib/python3.13/dist-packages/gym/core.py:317: DeprecationWarning: WARN: Initializing wrapper in old step API which returns one bool instead of two. It is re
  deprecation(
/usr/local/lib/python3.13/dist-packages/gym/wrappers/step_api_compatibility.py:39: DeprecationWarning: WARN: Initializing environment in old step API which returns one
  deprecation(
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a futu
  return datetime.utcnow().replace(tzinfo=utc)
```

The environment is shown below, where the letters (R, G, Y, B) represents the different locations and a tiny yellow colored rectangle is the taxi driving by our agent.

```
env.reset()
env.render()
```

```
/usr/local/lib/python3.13/dist-packages/gym/core.py:49: DeprecationWarning: WARN: You are calling render method, but you didn't specified the argument render_mode at e
If you want to render in human mode, initialize the environment in this way: gym.make('EnvName', render_mode='human') and don't call the render method.
See here for more information: https://www.gymlibrary.ml/content/api/
  deprecation(
/usr/local/lib/python3.13/dist-packages/pygame/pkgdata.py:25: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resource
  from pkg_resources import resource_stream, resource_exists
/usr/local/lib/python3.13/dist-packages/pkg_resources/__init__.py:3148: DeprecationWarning: Deprecated call to `pkg_resources.declare_namespace('sphinxcontrib')`.
Implementing implicit namespace packages (as specified in PEP 420) is preferred to `pkg_resources.declare_namespace`. See https://setuptools.pypa.io/en/latest/referenc
  declare_namespace(pkg)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a futu
  return datetime.utcnow().replace(tzinfo=utc)
```

Now, we initialize, Q table as a dictionary which stores state-action pair specifying value of performing an action a in state s.

```
q = {}
for s in range(env.observation_space.n):
    for a in range(env.action_space.n):
        q[(s,a)] = 0.0
```

We define a function called update_q_table which will update the Q values according to our Q learning update rule.

If you look at the below function, we take the value which has maximum value for a state-action pair and store it in a variable called qa, then we update the Q value of the preivous state by our update rule.

```
def update_q_table(prev_state, action, reward, nextstate, alpha, gamma):

    qa = max([q[(nextstate, a)] for a in range(env.action_space.n)])
    q[(prev_state,action)] += alpha * (reward + gamma * qa - q[(prev_state,action)])
```

Then, we define a function for performing epsilon-greedy policy. In epsilon-greedy policy, either we select best action with probability 1-epsilon or we explore new action with probability epsilon.

```
def epsilon_greedy_policy(state, epsilon):
    if random.uniform(0,1) < epsilon:
        return env.action_space.sample()
    else:
        return max(list(range(env.action_space.n)), key = lambda x: q[(state,x)])
```

Now we initialize necessary variables

alpha - TD learning rate

gamma - discount factor
epsilon - epsilon value in epsilon greedy policy

```
alpha = 0.4
gamma = 0.999
epsilon = 0.017
```

Now, Let us perform Q Learning!!!!

```
import numpy as np

# NumPy compatibility with older Gym versions
if not hasattr(np, "bool8"):
    np.bool8 = np.bool_

num_episodes = 8000

for i in range(num_episodes):
```

```
        r = 0

        prev_state = env.reset()

        while True:
            # Select action using epsilon-greedy policy
            action = epsilon_greedy_policy(prev_state, epsilon)

            # Perform action
            nextstate, reward, done, _ = env.step(action)

            # Update Q-table
            update_q_table(
                prev_state, action, reward, nextstate, alpha, gamma
            )

            prev_state = nextstate
            r += reward

            if done:
                break

        if (i + 1) % 100 == 0:
            print(f"Episode {i + 1}/{num_episodes}, Total reward: {r}")

    env.close()
```

```
Episode 100/8000, Total reward: -238
Episode 200/8000, Total reward: 11
Episode 300/8000, Total reward: -1
Episode 400/8000, Total reward: 5
Episode 500/8000, Total reward: 7
Episode 600/8000, Total reward: 11
Episode 700/8000, Total reward: 6
Episode 800/8000, Total reward: 10
Episode 900/8000, Total reward: 7
Episode 1000/8000, Total reward: 2
Episode 1100/8000, Total reward: 10
Episode 1200/8000, Total reward: 10
Episode 1300/8000, Total reward: 8
Episode 1400/8000, Total reward: 12
Episode 1500/8000, Total reward: 6
Episode 1600/8000, Total reward: 5
Episode 1700/8000, Total reward: 5
Episode 1800/8000, Total reward: 3
Episode 1900/8000, Total reward: 5
Episode 2000/8000, Total reward: 7
Episode 2100/8000, Total reward: 8
Episode 2200/8000, Total reward: 6
Episode 2300/8000, Total reward: 7
Episode 2400/8000, Total reward: 3
Episode 2500/8000, Total reward: 8
Episode 2600/8000, Total reward: 9
Episode 2700/8000, Total reward: 10
Episode 2800/8000, Total reward: 8
Episode 2900/8000, Total reward: -1
Episode 3000/8000, Total reward: 10
Episode 3100/8000, Total reward: 8
```

```
Episode 3200/8000, Total reward: 9
Episode 3300/8000, Total reward: 7
Episode 3400/8000, Total reward: 7
Episode 3500/8000, Total reward: 7
Episode 3600/8000, Total reward: 11
Episode 3700/8000, Total reward: 9
Episode 3800/8000, Total reward: 5
Episode 3900/8000, Total reward: 4
Episode 4000/8000, Total reward: 8
Episode 4100/8000, Total reward: 3
Episode 4200/8000, Total reward: 3
Episode 4300/8000, Total reward: 1
Episode 4400/8000, Total reward: 7
Episode 4500/8000, Total reward: -2
Episode 4600/8000, Total reward: 9
Episode 4700/8000, Total reward: 8
Episode 4800/8000, Total reward: 10
Episode 4900/8000, Total reward: 12
Episode 5000/8000, Total reward: 10
Episode 5100/8000, Total reward: 5
Episode 5200/8000, Total reward: 7
Episode 5300/8000, Total reward: 12
Episode 5400/8000, Total reward: 10
Episode 5500/8000, Total reward: 8
Episode 5600/8000, Total reward: 10
Episode 5700/8000, Total reward: 7
```

```
Start coding or generate with AI.
```